iCourse Subscriber 网页流程详解

本文档详细解析 icourse_subscriber 项目中涉及的所有网页请求流程，包括 WebVPN 代理机制、身份认证、课程 API 调用、视频 CDN 签名与下载等。

目录

1. 系统架构概览
2. WebVPN URL 编码机制
3. WebVPN IDP 登录认证（7 步）
4. iCourse CAS 认证（7 步）
5. 课程 API 调用
6. CDN 视频 URL 签名算法
7. 视频下载流程
8. 完整请求流转图

1. 系统架构概览

1.1 基本概念

项目通过复旦大学 WebVPN（ `webvpn.fudan.edu.cn` ）代理访问校内 iCourse 智慧教学平台（ `icourse.fudan.edu.cn` ），实现在校外环境下的课程视频下载。

1.2 涉及的服务

服务	域名	说明
WebVPN	`webvpn.fudan.edu.cn`	复旦大学 VPN 代理网关

服务	域名	说明
IDP	`id.fudan.edu.cn`	复旦大学统一身份认证平台
iCourse	`icourse.fudan.edu.cn`	智慧教学平台（课程、视频）
CDN	视频 CDN 服务器	视频文件存储与分发

1.3 请求流转总览

```
用户程序
  │
  ├─ 1. WebVPN IDP 登录 ──→ id.fudan.edu.cn（直连）
  │     获取 WebVPN session cookie
  │
  ├─ 2. iCourse CAS 认证 ──→ id.fudan.edu.cn（通过 WebVPN 代理）
  │     获取 iCourse 认证 cookie
  │
  ├─ 3. 课程 API 调用 ──→ icourse.fudan.edu.cn（通过 WebVPN 代理）
  │     获取课程列表、详情、视频地址
  │
  └─ 4. 视频下载 ──→ CDN 服务器（通过 WebVPN 代理）
        使用签名 URL 下载视频文件
```

1.4 关键配置

```python
# src/config.py
WEBVPN_BASE    = "https://webvpn.fudan.edu.cn"
IDP_BASE       = "https://id.fudan.edu.cn"
ICOURSE_BASE   = "https://icourse.fudan.edu.cn"
WEBVPN_AES_KEY = b"wrdvpnisthebest!"   # AES-128-CFB 密钥
WEBVPN_AES_IV  = b"wrdvpnisthebest!"   # AES-128-CFB 初始向量
TENANT_CODE    = "222"                   # 复旦大学租户 ID
GROUP_CODE     = "2095000001"
```

2. WebVPN URL 编码机制

2.1 原理

WebVPN 使用 **AES-128-CFB** 加密算法对目标域名进行编码，将原始 URL 转换为 WebVPN 代理 URL。所有通过 WebVPN 的请求都遵循此编码规则。

2.2 编码流程

```
 原始 URL:
  https://icourse.fudan.edu.cn/courseapi/v3/test

WebVPN URL:
  https://webvpn.fudan.edu.cn/https/77726476706e69737468656265737421f9f44e8935236d1e
                                  ├──────── IV hex ────────┤├──────── 加密后的域名
```

2.3 编码算法详解

输入: 原始 URL（如 `https://icourse.fudan.edu.cn/courseapi/v3/test` ）

步骤:

1. **解析 URL**: 提取协议（ `https` ）、域名（ `icourse.fudan.edu.cn` ）、端口、路径
2. **AES 加密域名**:
   - 密钥: `wrdvpnisthebest!` （16 字节）
   - IV: `wrdvpnisthebest!` （16 字节）
   - 模式: AES-128-CFB，segment_size=128
   - 输入: 域名的 UTF-8 编码
   - 输出: 加密结果的十六进制字符串
3. **构造 WebVPN URL**:

   ```
   {WEBVPN_BASE}/{协议}{端口后缀}/{IV_hex}{加密域名_hex}/{原始路径}
   ```

   - IV hex = `77726476706e69737468656265737421` （ `wrdvpnisthebest!` 的十六进制）
   - 端口后缀: 非标准端口时添加 `-{port}` ，标准端口（HTTP 80/HTTPS 443）省略

2.4 代码实现

```python
# src/webvpn.py
def encrypt_host(hostname: str) -> str:
    key = config.WEBVPN_AES_KEY        # b"wrdvpnisthebest!"
    iv = config.WEBVPN_AES_IV          # b"wrdvpnisthebest!"
    cipher = AES.new(key, AES.MODE_CFB, iv, segment_size=128)
    plaintext = hostname.encode("utf-8")
    encrypted = cipher.encrypt(plaintext)
    return hexlify(encrypted).decode("ascii")

def get_vpn_url(url: str) -> str:
    parsed = urlparse(url)
    encrypted = encrypt_host(parsed.hostname)
    iv_hex = hexlify(config.WEBVPN_AES_IV).decode("ascii")
    vpn_url = f"{config.WEBVPN_BASE}/{parsed.scheme}/{iv_hex}{encrypted}"
    if parsed.path:
        vpn_url += f"/{parsed.path.lstrip('/')}"
    return vpn_url
```

2.5 编码示例

原始域名	加密结果（hex）
`icourse.fudan.edu.cn`	`f9f44e8935236d1e781d8dad961b2631a501f26f`
`id.fudan.edu.cn`	（由 AES 动态计算）

3. WebVPN IDP 登录认证（7 步）

这是建立 WebVPN 会话的第一步。通过复旦大学 IDP（统一身份认证）完成登录，获取 `wengine_vpn_ticket` cookie。

注意: 此流程中所有请求 **直连** IDP 服务器（ `id.fudan.edu.cn` ），不经过 WebVPN 代理。

步骤 1: 获取认证上下文（Get Auth Context）

```
GET https://id.fudan.edu.cn/idp/authCenter/authenticate
    ?service=https%3A%2F%2Fwebvpn.fudan.edu.cn%2Flogin%3Fcas_login%3Dtrue
```

说明: 访问 IDP 认证入口，服务参数指定回调地址为 WebVPN 登录页。

响应: HTTP 302 重定向链，最终 Location 头中包含 `lck` 参数。

```
302 → Location: https://id.fudan.edu.cn/ac/?lck=XXXXXXXXXX&entityId=...
```

提取: `lck` （login context key）和 `entity_id` （设为 `WEBVPN_BASE` ）

步骤 2: 查询认证方式（Query Auth Methods）

```
POST https://id.fudan.edu.cn/idp/authn/queryAuthMethods
Content-Type: application/json
Referer: https://id.fudan.edu.cn/ac/

{
    "lck": "...",
    "entityId": "https://webvpn.fudan.edu.cn"
}
```

响应:

```
{
    "code": "200",
    "requestType": "chain_type",
    "data": [
        {
            "moduleCode": "userAndPwd",
            "authChainCode": "XXXXXXXX-XXXX-..."
        },
        ...
    ]
}
```

提取: `authChainCode` （选择 `moduleCode == "userAndPwd"` 的条目）和 `requestType`

步骤 3: 获取 RSA 公钥（Get Public Key）

```
GET https://id.fudan.edu.cn/idp/authn/getJsPublicKey
Referer: https://id.fudan.edu.cn/ac/
```

响应:

```
{
    "code": "200",
    "data": "MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQ..."   // Base64 编码的 RSA 公钥
}
```

提取: RSA 公钥的 Base64 字符串

步骤 4: 加密密码（Encrypt Password）

使用 RSA 公钥（PKCS1_v1_5 padding）加密用户明文密码。

```
# 构造 PEM 格式
pem = "-----BEGIN PUBLIC KEY-----\n" + pub_key_b64 + "\n-----END PUBLIC KEY-----"
rsa_key = RSA.import_key(pem)
cipher = PKCS1_v1_5.new(rsa_key)
encrypted = cipher.encrypt(password.encode("utf-8"))
encrypted_password = base64.b64encode(encrypted).decode("ascii")
```

输出: Base64 编码的加密密码字符串

步骤 5: 执行认证（Auth Execute）

```
POST https://id.fudan.edu.cn/idp/authn/authExecute
Content-Type: application/json
Referer: https://id.fudan.edu.cn/ac/
Origin: https://id.fudan.edu.cn

{
    "authModuleCode": "userAndPwd",
    "authChainCode": "...",
    "entityId": "https://webvpn.fudan.edu.cn",
    "requestType": "chain_type",
    "lck": "...",
    "authPara": {
        "loginName": "学号",
        "password": "RSA加密后的密码(Base64)",
        "verifyCode": ""
    }
}
```

响应:

```
{
    "code": "200",
    "loginToken": "XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX",
    ...
}
```

提取: `loginToken`

步骤 6: 获取 CAS Ticket（Get CAS Ticket）

```
POST https://id.fudan.edu.cn/idp/authCenter/authnEngine
Content-Type: application/x-www-form-urlencoded
Referer: https://id.fudan.edu.cn/ac/

loginToken=XXXXXXXX-XXXX-...
```

响应: HTML 页面，包含 JavaScript 重定向代码：

```
<script>
var locationValue = "https://webvpn.fudan.edu.cn/login?cas_login=true&ticket=ST-XXXX
window.location.href = locationValue;
</script>
```

提取: 含 `ticket=` 参数的 URL（通过正则从 HTML 中匹配）

步骤 7: 建立 WebVPN 会话（Establish Session）

```
GET https://webvpn.fudan.edu.cn/login?cas_login=true&ticket=ST-XXXXX-XXXXX
```

说明: 携带 CAS ticket 访问 WebVPN 登录页。WebVPN 验证 ticket 后设置会话 cookie。

关键 Cookie: `wengine_vpn_ticket` ——后续所有 WebVPN 代理请求都依赖此 cookie。

注意: 此步骤可能较慢（超时可达 90 秒）。代码实现了最多 3 次重试，并在超时时检查 cookie 是否已设置。

4. iCourse CAS 认证（7 步）

WebVPN 登录后，还需要单独对 iCourse 平台进行 CAS 认证，获取 iCourse 的会话 cookie。

注意: 此流程中所有请求 **通过 WebVPN 代理**（URL 经过 WebVPN 编码）。

步骤 1: 发起 CAS 登录（Initiate via casapi）

```
GET [WebVPN编码] https://icourse.fudan.edu.cn/casapi/index.php
    ?r=auth/login
    &school_login=1
    &tenant_code=222
    &forward=https%3A%2F%2Ficourse.fudan.edu.cn%2F
```

说明: 模拟浏览器中点击"校内用户登录"按钮。casapi 会生成正确的 service URL 并 302 重定向到 IDP。

响应: HTTP 302 重定向链（最多跟踪 15 次），最终到达 IDP 登录页面，URL 中包含 `lck` 参数。

提取: `lck` ; `entity_id` 设为 `ICOURSE_BASE` （ `https://icourse.fudan.edu.cn` ）

已知问题: 重定向链有时会落在 WebVPN 登录页面而非 IDP，导致无法提取 `lck` 。需重试。

步骤 2: 查询认证方式（Query Auth Methods）

```
POST [WebVPN编码] https://id.fudan.edu.cn/idp/authn/queryAuthMethods
Content-Type: application/json
Referer: [WebVPN编码的 IDP /ac/ 页面]
Origin: https://webvpn.fudan.edu.cn

{
    "lck": "...",
    "entityId": "https://icourse.fudan.edu.cn"
}
```

与 WebVPN 登录流程类似，但 `entityId` 为 iCourse 地址。

提取: `authChainCode` 和 `requestType`

步骤 3: 获取 RSA 公钥

```
GET [WebVPN编码] https://id.fudan.edu.cn/idp/authn/getJsPublicKey
Referer: [WebVPN编码的 IDP /ac/ 页面]
```

步骤 4: 加密密码

与 WebVPN 登录步骤 4 相同。

步骤 5: 执行认证

```
POST [WebVPN编码] https://id.fudan.edu.cn/idp/authn/authExecute
Content-Type: application/json

{
    "authModuleCode": "userAndPwd",
    "authChainCode": "...",
    "entityId": "https://icourse.fudan.edu.cn",
    "requestType": "chain_type",
    "lck": "...",
    "authPara": {
        "loginName": "学号",
        "password": "RSA加密后的密码",
        "verifyCode": ""
    }
}
```

提取: `loginToken`

步骤 6: 获取 CAS Ticket

```
POST [WebVPN编码] https://id.fudan.edu.cn/idp/authCenter/authnEngine

loginToken=XXXXXXXX-XXXX-...
```

响应: HTML 中包含 ticket URL，形如：

```
https://icourse.fudan.edu.cn/casapi/index.php?forward=...&r=auth/login&ticket=ST-XX
```

注意: URL 可能已被 WebVPN 重写为代理格式，也可能是原始格式（需检查并转换）。

步骤 7: 跟随 Ticket 完成认证

```
GET [WebVPN编码] https://icourse.fudan.edu.cn/casapi/index.php?...&ticket=ST-XXXXX
```

说明: 携带 ticket 访问 iCourse casapi，casapi 验证 ticket 后设置 iCourse 会话 cookie。

验证: 调用 `/eduuserapi/v1/infosimple` 接口确认登录成功：

```
GET [WebVPN编码] https://icourse.fudan.edu.cn/eduuserapi/v1/infosimple
```

成功响应:

```
{
    "code": 0,
    "data": {
        "realname": "张三",
        ...
    }
}
```

5. 课程 API 调用

以下所有 API 请求均通过 WebVPN 代理（自动 URL 编码）。

5.1 获取用户信息（Get Userinfo）

```
GET https://icourse.fudan.edu.cn/userapi/v1/infosimple
```

响应:

```
{
    "code": 200,
    "params": {
        "id": 93702,
        "tenant_id": 222,
        "phone": "13110098269",
        "account": "学号"
    }
}
```

注意:

- 此接口返回 `code: 200` （非 `0` ），数据在 `params` 字段中（非 `data` ）
- 返回值用于后续的 CDN 视频签名

5.2 获取课程列表（Get Course List）

```
GET https://icourse.fudan.edu.cn/portal/courseapi/v3/multi-search/get-course-list
    ?tenant=222
    &title=
    &term=24
    &kkxy_code=
    &course_type=
    &course_student_type=
    &page=1
    &per_page=20
```

响应:

```
{
    "code": 0,
    "data": {
        "total": 150,
        "list": [
            {
                "id": 30315,
                "title": "数据结构",
                "realname": "李老师",
                ...
            },
            ...
        ]
    }
}
```

5.3 获取课程详情（Get Course Detail）

```
GET https://icourse.fudan.edu.cn/courseapi/v3/multi-search/get-course-detail
    ?course_id=30315
```

响应:

```
{
    "code": 0,
    "data": {
        "title": "数据结构",
        "realname": "李老师",
        "sub_list": {
            "2024": {
                "09": {
                    "05": [
                        {
                            "id": 123456,
                            "sub_title": "第1讲 绪论",
                            "lecturer_name": "李老师"
                        }
                    ]
                }
            }
        }
    }
}
```

sub_list **结构**: 三层嵌套字典 {年: {月: {日: [课次列表]}}}

提取逻辑:

```
for year, months in sub_list.items():
    for month, days in months.items():
        for day, items in days.items():
            for item in items:
                # item["id"] = sub_id
                # item["sub_title"] = 课次标题
```

5.4 获取课次详情（Get Sub Detail）

```
GET https://icourse.fudan.edu.cn/courseapi/v3/multi-search/get-sub-detail
    ?course_id=30315
    &sub_id=123456
```

响应: 包含课次内容信息，其中可能有 `content.playback.url` （视频地址），但 **此 URL 未签名**，无法直接下载。

5.5 获取课次信息（Get Sub Info）——关键接口

```
GET https://icourse.fudan.edu.cn/courseapi/v3/portal-home-setting/get-sub-info
    ?course_id=30315
    &sub_id=123456
```

响应:

```
{
    "code": 0,
    "data": {
        "now": 1772348134,
        "playurl": {
            "0": "https://cdn.example.com/video/path/file.mp4",
            "1": "https://cdn.example.com/video/path/file_hd.mp4",
            "4": "https://cdn.example.com/video/path/file_sd.mp4",
            "now": 1772348134
        },
        "video_list": {
            "0": {
                "preview_url": "https://cdn.example.com/video/path/file.mp4",
                ...
            }
        }
    }
}
```

关键字段:

- `now` ：服务器时间戳，用于 CDN 签名
- `playurl` ：视频地址字典，key 为清晰度索引（ `"0"` ， `"1"` ， `"4"` ），`"now"` 为时间戳
- `video_list` ：另一种格式的视频列表， `preview_url` 中包含视频地址

URL 提取优先级:

1. `video_list[*].preview_url` （优先使用，无 `/0/` 前缀）

2. `playurl[key]` （备选，可能有 `/0/` 前缀）
3. `get-sub-detail` 的 `content.playback.url` （最后手段，未签名）

5.6 获取字幕/转录（Get Transcript）

```
GET https://icourse.fudan.edu.cn/courseapi/v3/web-socket/search-trans-result
    ?sub_id=123456
    &format=json
```

响应:

```
{
    "code": 0,
    "list": [
        {
            "all_content": [
                {
                    "BeginSec": 0.5,
                    "Text": "同学们好，今天我们来学习..."
                },
                {
                    "BeginSec": 5.2,
                    "Text": "数据结构是计算机科学的核心课程..."
                }
            ]
        }
    ]
}
```

处理: 按 `BeginSec` 排序后拼接所有 `Text` 字段。

6. CDN 视频 URL 签名算法

6.1 背景

从 API 获取的视频 URL 是未签名的原始地址，直接访问会返回 403。需要添加 CDN 认证参数 `clientUUID` 和 `t` 才能下载。

6.2 签名算法来源

算法逆向自 iCourse 前端 JavaScript 代码：

- **文件**: `0.c6f283b4c2a6f87c4fa0.js`
- **模块**: `"1P4N"`
- **触发条件**: `window.CONFIG.OPEN_PLAY_AUTH === true`

6.3 算法详解

输入参数:

参数	来源	示例值
`video_url`	get-sub-info API	`https://cdn.example.com/video/xxx.mp4`
`user_id`	userinfo API → `id`	`93702`
`tenant_id`	userinfo API → `tenant_id`	`222`
`phone`	userinfo API → `phone`	`13110098269`
`now`	get-sub-info API → `now`	`1772348134`

签名步骤:

1. 提取 URL 路径名

   pathname = urlparse(video_url).path

   例: "/video/path/file.mp4"

2. 反转手机号

   reversed_phone = phone[::-1]

   例: "96289001131"

3. 拼接哈希输入

   hash_input = pathname + user_id + tenant_id + reversed_phone + timestamp

   例: "/video/path/file.mp4" + "93702" + "222" + "96289001131" + "1772348134"

4. 计算 MD5 哈希

   md5_hash = md5(hash_input.encode()).hexdigest()

   例: "0a1353d85a01281028a0863602696c1e"

5. 构造 t 参数

   t = f"{user_id}-{timestamp}-{md5_hash}"

   例: "93702-1772348134-0a1353d85a01281028a0863602696c1e"

6. 生成 clientUUID（随机 UUID v4）

   clientUUID = str(uuid.uuid4())

   例: "a1b2c3d4-e5f6-7890-abcd-ef1234567890"

7. 拼接最终签名 URL

   signed_url = f"{video_url}?clientUUID={clientUUID}&t={t}"

6.4 JavaScript 原始代码（反混淆后）

```javascript
// 模块 "1P4N" 中的签名函数
function signUrl(url, isLive, userId, tenantId, phone, liveInfo) {
    if (!window.CONFIG.OPEN_PLAY_AUTH) return url;

    var pathname = new URL(url).pathname;
    var timestamp = liveInfo.now || Math.floor(Date.now() / 1000);
    var reversedPhone = phone.split("").reverse().join("");
    var hashInput = pathname + userId + tenantId + reversedPhone + timestamp;
    var md5Hash = md5(hashInput);        // a() 即 MD5 函数
    var t = userId + "-" + timestamp + "-" + md5Hash;

    var separator = url.includes("?") ? "&" : "?";
    return url + separator + "clientUUID=" + uuid() + "&t=" + t;
}
```

调用处（播放器组件）：

```javascript
Object(u.a)(
    t || this.liveUrl,     // video_url
    c,                      // isLive
    this.userinfo.id,       // user_id
    this.userinfo.tenant_id,// tenant_id
    this.userinfo.phone,    // phone
    this.liveInfo           // { now: timestamp, ... }
)
```

6.5 Python 实现

```python
# src/icourse.py - ICourseClient.sign_video_url()
def sign_video_url(self, video_url: str, now: int | None = None) -> str:
    userinfo = self.get_userinfo()
    user_id = userinfo.get("id", "")
    tenant_id = userinfo.get("tenant_id", "")
    phone = str(userinfo.get("phone", ""))

    if now is None:
        now = int(time.time())

    reversed_phone = phone[::-1]
    pathname = urlparse(video_url).path

    hash_input = f"{pathname}{user_id}{tenant_id}{reversed_phone}{now}"
    md5_hash = hashlib.md5(hash_input.encode()).hexdigest()
    t_param = f"{user_id}-{now}-{md5_hash}"

    client_uuid = str(uuid.uuid4())
    sep = "&" if "?" in video_url else "?"
    return f"{video_url}{sep}clientUUID={client_uuid}&t={t_param}"
```

7. 视频下载流程

7.1 完整下载流程

```
1. 登录
   WebVPN IDP 登录（7 步）
        ↓
   iCourse CAS 认证（7 步）

2. 获取视频地址
   get_userinfo()     → 获取 user_id, tenant_id, phone
        ↓
   get_sub_info()     → 获取未签名视频 URL + 服务器时间戳 now
        ↓
   sign_video_url()   → 添加 clientUUID 和 t 参数

3. 下载视频
   vpn.get(signed_url, stream=True)
        ↓
   iter_content(chunk_size=8192) → 写入文件
```

7.2 请求详情

获取签名 URL

```python
client = ICourseClient(vpn_session)

# 1. 获取用户信息（自动缓存）
userinfo = client.get_userinfo()
# → GET https://icourse.fudan.edu.cn/userapi/v1/infosimple

# 2. 获取视频 URL（内部调用 get_sub_info + sign_video_url）
video_url = client.get_video_url(course_id="30315", sub_id="123456")
# → GET https://icourse.fudan.edu.cn/courseapi/v3/portal-home-setting/get-sub-info
# → 签名处理
```

下载视频

```
# 3. 下载（通过 WebVPN 代理）
client.download_video(video_url, output_path="output/video.mp4")
```

实际 HTTP 请求:

```
GET [WebVPN编码] https://cdn.example.com/video/xxx.mp4?clientUUID=...&t=93702-177234

Headers:
  Cookie: wengine_vpn_ticket=...  (WebVPN 会话)
  User-Agent: Mozilla/5.0 ...

Response:
  Status: 200
  Content-Type: video/mp4
  Content-Length: 314572800  (约 300MB)
```

7.3 WebVPN URL 的特殊处理

下载方法根据 URL 是否已经是 WebVPN 格式选择不同的请求方式：

```
def download_video(self, video_url, output_path, chunk_size=8192):
    if video_url.startswith(config.WEBVPN_BASE):
        # 已经是 WebVPN URL，直接请求
        resp = self.vpn.get_raw(video_url, stream=True, timeout=300)
    else:
        # 原始 URL，需要 WebVPN 编码后请求
        resp = self.vpn.get(video_url, stream=True, timeout=300)
```

8. 完整请求流转图

8.1 时序图

```
用户程序                    WebVPN                    IDP                    iCourse
  │                         │                         │                        │
  │ ═══════════ 阶段 1: WebVPN 登录（直连 IDP）═══════════                      │
  │                         │                         │                        │
  ├──GET authenticate───────┼────────────────────────→│                        │
  │←─302 (lck)──────────────┼─────────────────────────│                        │
  ├──POST queryAuthMethods──┼────────────────────────→│                        │
  │←─200 (authChainCode)────┼─────────────────────────│                        │
  ├──GET getJsPublicKey─────┼────────────────────────→│                        │
  │←─200 (RSA pubkey)───────┼─────────────────────────│                        │
  │  [本地RSA加密密码]       │                         │                        │
  ├──POST authExecute───────┼────────────────────────→│                        │
  │←─200 (loginToken)───────┼─────────────────────────│                        │
  ├──POST authnEngine───────┼────────────────────────→│                        │
  │←─HTML (ticket URL)──────┼─────────────────────────│                        │
  ├──GET ticket URL─────────┼→                        │                        │
  │←─200 (session cookie)───│                         │                        │
  │                         │                         │                        │
  │ ═══════════ 阶段 2: iCourse CAS 认证（通过 WebVPN）═══════════════          │
  │                         │                         │                        │
  ├──GET casapi/login───────┼→ 代理 ──────────────────┼────────────────────────→│
  │←─302 chain (lck)────────┼←────────────────────────┼─────────────────────────│
  ├──POST queryAuthMethods──┼→ 代理 ──────────────────→│                        │
  │←─200 (authChainCode)────┼←────────────────────────│                        │
  ├──GET getJsPublicKey─────┼→ 代理 ──────────────────→│                        │
  │←─200 (RSA pubkey)───────┼←────────────────────────│                        │
  │  [本地RSA加密密码]       │                         │                        │
  ├──POST authExecute───────┼→ 代理 ──────────────────→│                        │
  │←─200 (loginToken)───────┼←────────────────────────│                        │
  ├──POST authnEngine───────┼→ 代理 ──────────────────→│                        │
  │←─HTML (ticket URL)──────┼←────────────────────────│                        │
  ├──GET casapi?ticket=... ─┼→ 代理 ──────────────────┼────────────────────────→│
  │←─200 (iCourse cookie)───┼←────────────────────────┼─────────────────────────│
  │                         │                         │                        │
  │ ═══════════ 阶段 3: 课程 API 调用（通过 WebVPN）═══════════════            │
  │                         │                         │                        │
  ├──GET userinfo───────────┼→ 代理 ──────────────────┼────────────────────────→│
```

```
│←200 (id,tenant,phone)─┼←──────────────────────┼──────────────────────────│
├──GET get-sub-info────────┼→ 代理 ────────────────┼──────────────────────────→│
│←200 (playurl, now)─────┼←──────────────────────┼──────────────────────────│
│  [本地CDN签名]          │                       │                          │
│                        │                       │                          │
│ ═══════════ 阶段 4: 视频下载（通过 WebVPN）═══════════════════════          │
│                        │                       │                          │
├──GET video.mp4?t=...───┼→ 代理 ────────────────┼──────────────────────────────
│←200 (video stream)─────┼←──────────────────────┼──────────────────────────────
│                        │                       │                          │
```

8.2 认证状态汇总

阶段	获取的凭证	存储方式	用途
WebVPN 登录	`wengine_vpn_ticket`	Session Cookie	WebVPN 代理访问
iCourse CAS	iCourse 会话 cookie	Session Cookie	iCourse API 调用
CDN 签名	`clientUUID` + `t` 参数	URL 查询参数	视频文件下载

8.3 错误处理与重试

场景	错误表现	处理策略
WebVPN 登录超时	`requests.exceptions.Timeout`	最多 3 次重试，检查 cookie 是否已设置
iCourse CAS 重定向异常	无法提取 `lck`	整体重试（重新 WebVPN 登录 + CAS 认证）
视频 URL 签名错误	HTTP 403	检查 userinfo 参数和时间戳
视频未签名	HTTP 403/500	确认使用 `get-sub-info` 而非 `get-sub-detail`

附录

A. WebVPN Session 的 HTTP 方法封装

```python
class WebVPNSession:
    def get(self, url, **kwargs):
        """自动将原始 URL 转换为 WebVPN URL 后发送 GET 请求"""
        vpn_url = get_vpn_url(url)
        return self.session.get(vpn_url, **kwargs)

    def get_raw(self, url, **kwargs):
        """直接发送 GET 请求，不做 URL 转换（用于已编码的 URL）"""
        return self.session.get(url, **kwargs)
```

B. 项目文件结构

```
icourse_subscriber/
├── src/
│   ├── config.py       # 配置常量（URL、密钥、环境变量）
│   ├── webvpn.py       # WebVPN URL 编码 + IDP 认证 + CAS 认证
│   └── icourse.py      # iCourse API 客户端（课程、签名、下载）
├── test_login.py       # 集成测试：登录 + API 调用
├── test_download.py    # 集成测试：视频签名 + 下载
└── debug_sub_detail.py # 调试工具：API 响应检查
```